Control Statements

- Control statements in C language are instructions that determine the flow of a program's execution based on certain conditions or repetitions.
- They help decide whether a block of code should run, repeat, or skip.
- These statements make programs dynamic by adding decision-making, looping, and jumping capabilities, which are essential for solving real-world problems.

Types of Control Statements in C:

There are three types of control statements in C programming:

1. **Decision Making (or) Selection (or) Branching Control Statements:**
   These statements allow the program to make decisions and execute specific blocks of code based on conditions.
2. **Looping (or) Repetitive Control Statements**:
   These statements repeatedly execute a block of code as long as a specified condition is true.
3. **Jump Statements:** These statements transfer the program's control from one part of the code to another, either conditionally or unconditionally.

---

1. Decision Making (or) Selection (or) Branching Control Statements

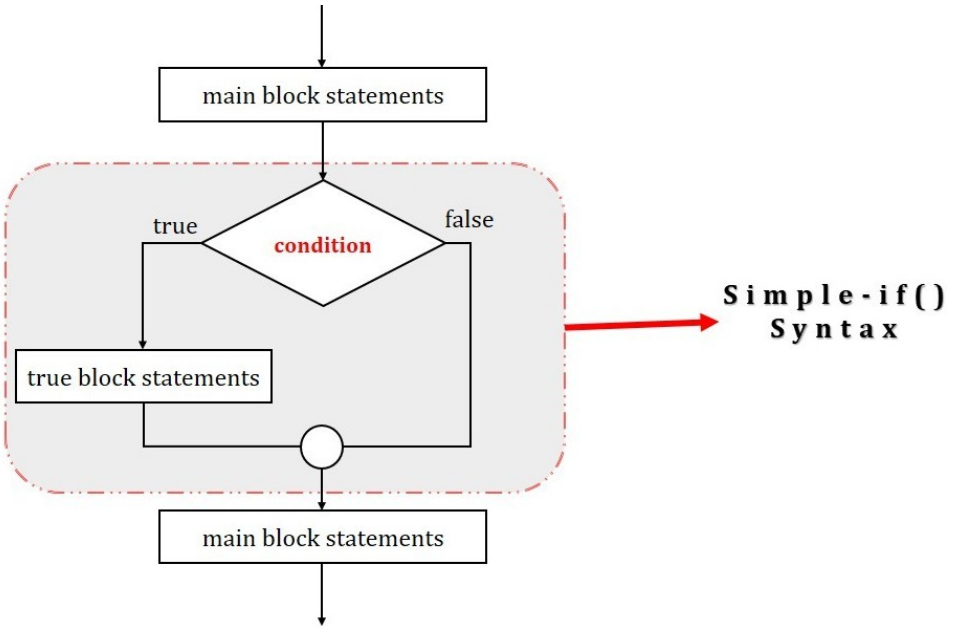
These statements in C allow the program to make choices and execute specific blocks of code based on conditions.

i. if()/Simple-if() statement:

- ✓ The if() statement in C executes a block of code only if the given condition is true.
- ✓ ***Syntax:***

```
// Optional main block statements;
if(<condition>){
   //Statements to execute if condition is true;
}
// Optional main block statements;
```

- ✓ ***Flow Representation:***



- ✓ ***Example:***

```
// program to check a number is positive
#include <stdio.h>
int main() {
   int num;
   printf("Enter an integer:");
   scanf("%d",&num);
   if (num > 0) {
      printf("The number %d is positive.",num);
   }
   printf("\nGood Bye...!");
   return 0;
}
```

- ✓ Output-1:

  Enter an integer: 12
  The number 12 is positive.
  Good Bye…!

- ✓ Output-2:

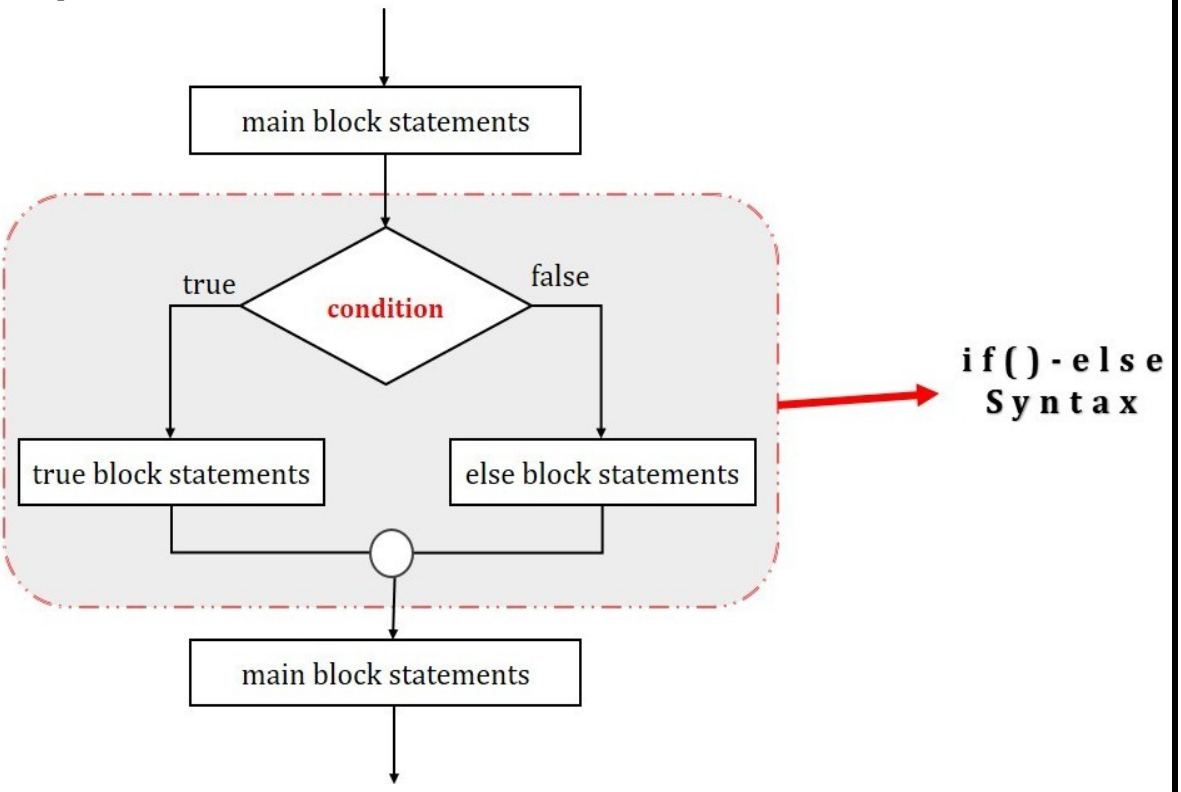
  Enter an integer: -45
  Good Bye…!

ii. if()-else:

- ✓ The if-else statement in C contains two block of statements i.e., true block statements and else block statements.
- ✓ It executes one block of code, if the condition is true then it executes true block statements otherwise executes else block statements.
- ✓ ***Syntax:***

```
// Optional main block statements;
if(<condition>){
   //Statements to execute if condition is true;
}
else{
   //Statements to execute if condition is false;
}
 // Optional main block statements;
```

- ✓ ***Flow Representation:***



- ***Example:***

```
// program to check a number is even or odd
#include <stdio.h>
int main() {
   int num;
   printf("Enter an integer:");
   scanf("%d",&num);
   if (num%2 == 0) {
      printf("The number %d is EVEN.",num);
   }

else{
    printf("The number %d is ODD.",num);
}
   printf("\nGood Bye...!");
   return 0;
}
```

- Output-1:

```
Enter an integer: 124
The number 12 is EVEN
Good Bye...!
```

- Output-2:

```
Enter an integer: 971
The number 9 is ODD
Good Bye...!
```

iii. Nested-if():

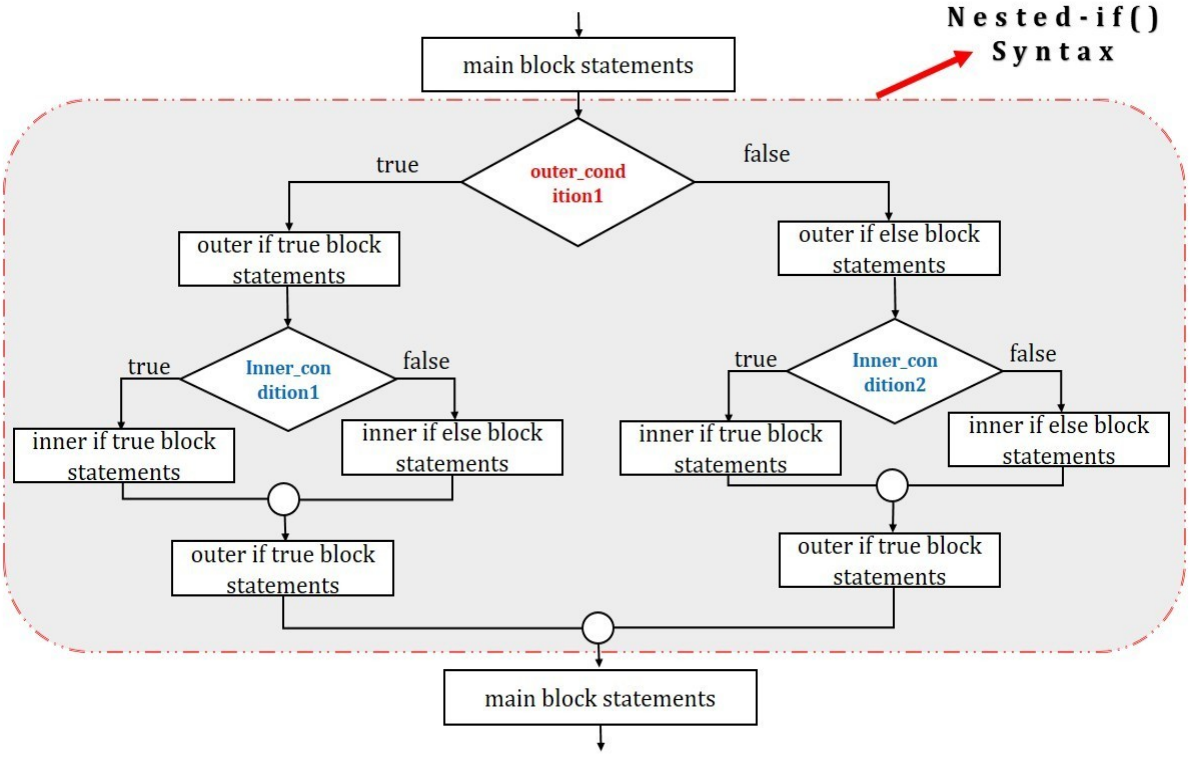
- It is an if() statement inside another if() statement, used to check multiple conditions.
- If we have if-else statement within either the body of if statement or the body of else statement or in the body of both if and else, then this is known as nesting of if else statement.

✓ ***Syntax:***

```
// Optional main block statements;
if(<outer-condition>){
   //Optional statements to execute if outer-condition is true;
    if(<inner-condition_1>){
          //Statements to execute if inner-condition_1 is true;
     }
     else{
          //Statements to execute if inner-condition_1 is false;
     }
 }
 else{
   //Optional statements to execute if outer-condition is false;
       if(<inner-condition_2>){
           //Statements to execute if inner-condition_2 is true;
        }
       else{
           //Statements to execute if inner-condition_2 is false;
        }
}
// Optional main block statements;
```

Optional block

✓ ***Flow Representation:***



✓ ***Example:***

```
// program to check the largest of three values
#include <stdio.h>
int main() {
   int num1,num2,num3;
   printf("Enter three integer value:");
   scanf("%d%d%d",&num1,&num2,&num3);
   if (num1>num2) {
      if(num1>num3){
         printf("%d is larger than %d and %d",num1,num2,num3);
      }
      else{
          printf("%d is larger than %d and %d",num3,num1,num2);
      }
   }
else{
     if(num2>num3){
         printf("%d is larger than %d and %d",num2,num1,num3);
      }
      else{
          printf("%d is larger than %d and %d",num3,num1,num2);
      }
}
   printf("\nGood Bye...!");
   return 0;
}
```

✓ Output-1:

```
Enter three integer value: 12
28
16
28 is larger than 12 and 16
Good Bye...!
```

✓ Output-2:

```
Enter three integer value: 45
28
16
45 is larger than 28 and 16
Good Bye...!
```

- ✓ Output-3:

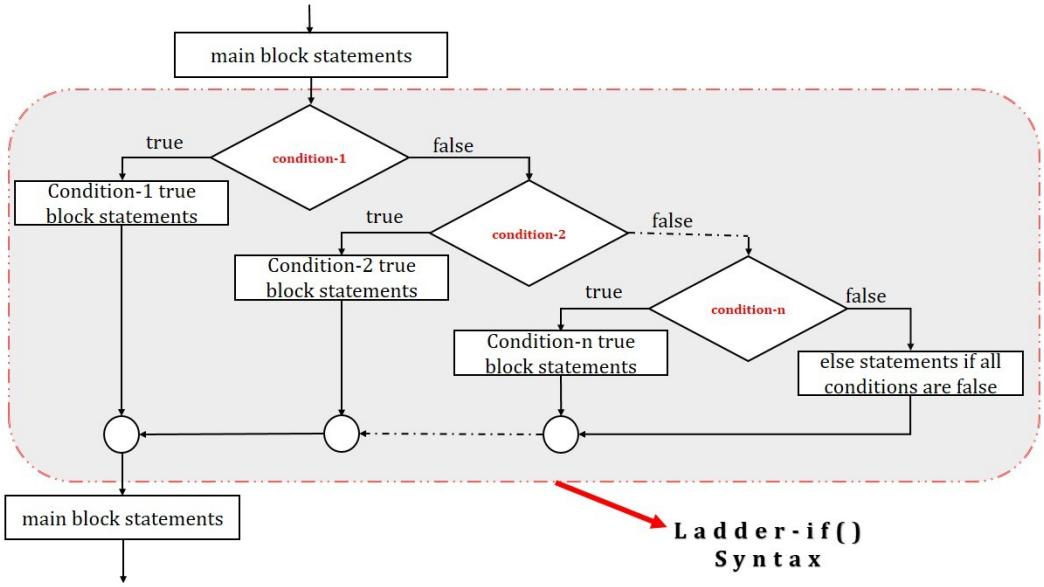
  Enter three integer value: 45
  28
  58
  58 is larger than 45 and 28
  Good Bye…!

iv. if()-else-if() or Ladder-if():

- ✓ This conditional statement in C checks multiple conditions one by one until a true condition is found.
- ✓ This is a type of nesting in which there is an if-else statement in every else part except the last else part.
- ✓ ***Syntax:***

```
//Optional main block statements;
if (<condition-1>) {
   // Code if condition-1 is true;
} else if (<condition-2>) {
   // Code if condition-2 is true;
}
----------
if(<condition-n>){
  //Code if condition-n is true;
}
else {
   // Code if none of the conditions are true;
}
//Optional main block statements;
```

- ✓ ***Flow Representation:***



- ***Example:***

```
//program to find the grade of the student
#include <stdio.h>
int main() {
   float per;
   printf("Enter student percentage");
   scanf("%f",&per);
   if (per >=90 ) {
      printf("Student passed in GRADE-A+ with %f percentage", per);
   }
  else if (per >= 80) {
      printf("Student passed in GRADE-A with %f percentage", per);
   }
  else if (marks >= 70) {
      printf("Student passed in GRADE-B with %f percentage", per);
   }
  else if (marks >= 60) {
      printf("Student passed in GRADE-C with %f percentage", per);
   }
 else if (marks >= 50) {
      printf("Student passed in GRADE-D with %f percentage", per);
   }
  else {
      printf("Student failed with %f percentage", per);
   }
  printf("\nGood Bye...!");
   return 0;
}
```

- Output-1:

```
Enter student percentage: 94
Student passed in GRADE-A+ with 94 percentage
Good Bye...!
```

- Output-2:

```
Enter student percentage: 89
Student passed in GRADE-A with 89 percentage
Good Bye...!
```

- Output-3:

```
Enter student percentage: 52
Student passed in GRADE-D with 73 percentage
Good Bye...!
```

- ✓ Output-4:
  Enter student percentage: 49
  Student failed with 49 percentage
  Good Bye…!

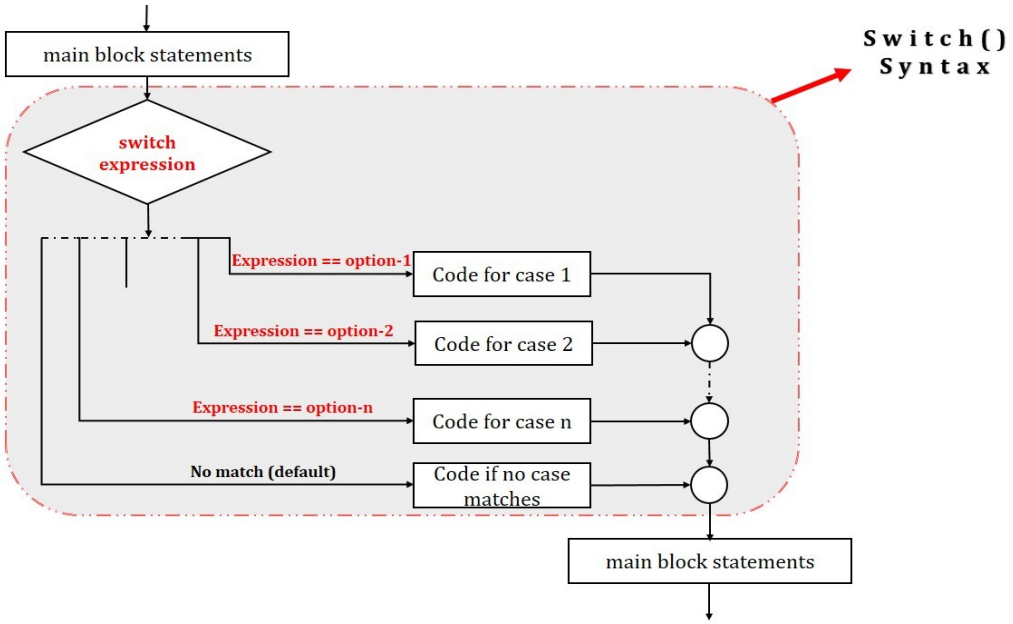
v. switch():

- ✓ The switch case in C tests an expression against multiple values (cases) and executes the matching block of code.
- ✓ ***Components in switch:***
  - switch – keyword used to begin multi-way selection statement.
  - expression –an expression whose value determines which case is selected.
  - case - keyword use to define a possible matching value.
  - option – it is a constant integer value or constant expression associated with a case.
  - break – terminates the switch statement and transfers control outside it.
  - default – executes when no case value matches the expression
  - { } – enclose the body of the switch

  Important: The break and default statements are optional.
- ✓ ***Rules of switch:***
  - Rule-1: switch expression must have int, chat or enumeration type. Floats and string are not acceptable.
  - Rule-2: options must be constant expression
  - Rule-3: duplicate case options are not allowed
- ✓ ***Syntax:***

```
// optional main block statements;
switch (<expression>) {
   case <option-1>:
      // Code for case 1;
      break;
   case <option-2>:
      // Code for case 2;
      break;
   ……..
    case <option-n>:
      // Code for case 2;
      break;
   default:
      // Code if no case matches;
}
// optional main block statements;
```

✓ ***Flow Representation:***



✓ ***Example:***

```
//program to print day of the week
#include <stdio.h>
int main() {
   int day;
   printf("Enter your choice from 1 to 7:");
   scanf("%d",&day);
   switch (day) {
      case 1:
         printf("Your day is Sunday...!");
         break;
      case 2:
         printf("Your day is Monday...!");
         break;
      case 3:
         printf("Your day is Tuesday...!");
         break;
      case 4:
         printf("Your day is Wednesday...!");
         break;
      case 5:
         printf("Your day is Thursday...!");
         break;
      case 6:
         printf("Your day is Friday...!");
         break;
```

```
        case 7:
            printf("Your day is Saturday…!");
            break;
        default:
            printf("Sorry, Invalid option…!");
    }
    printf("\nGood Bye…!");
    return 0;
}
```

- ✓ Output-1:
  Enter your choice from 1 to 7:6
  Your day is Friday…!
  Good Bye…!
- ✓ Output-2:
  Enter your choice from 1 to 7:2
  Your day is Monday…!
  Good Bye…!
- ✓ Output-3:
  Enter student percentage: 8
  Sorry, Invalid option…!
  Good Bye…!

❒ Important Points:

- The condition inside if() need not be a relational expression.
- Assignment (=) and equality (==) are different.
- A semicolon immediately after if() creates an empty statement.
- Braces {} are optional for a single statement.
- Without braces, only the immediately following statement belongs to the if().
- An else is associated with the nearest unmatched if.
- An else cannot appear independently.
- The else-if ladder checks conditions sequentially.
- A nested-if() allows statements to execute between condition checks, whereas an ladder-if() checks the next condition directly when the previous condition is false.

2. Iterative (or) Repetitive Control Statements

Loop control statements in C are used to repeatedly execute a block of code as long as a specific condition is true. The three important components of a loop are:

- ✓ ***Initialization:***
  - Initialization is the process of assigning an initial value to a loop control variable before starting the execution of a loop.
  - It specifies the starting value of the loop control variable.
  - It is generally executed only once before the loop begins.
  - It determines where the repetition starts.
  - Without proper initialization, the loop may produce unexpected results.
- ✓ ***Test Condition (Condition Checking):***
  - A test condition is a logical or relational expression that determines whether the loop body should execute or terminate.
  - The condition is evaluated as either true (nonzero) or false (zero).
  - If the condition is true, the loop body executes.
  - If the condition is false, the loop terminates.
  - In for and while loops, the condition is checked before executing the loop body.
  - In a do-while loop, the condition is checked after executing the loop body.
- ✓ ***Updation:***
  - Updation is the process of modifying the value of the loop control variable after each iteration to control the repetition of the loop.
  - It changes the value of the loop control variable.
  - It generally uses increment or decrement operations.
  - It helps the loop progress toward its termination condition.
  - Incorrect updation may result in an infinite loop.

Example:

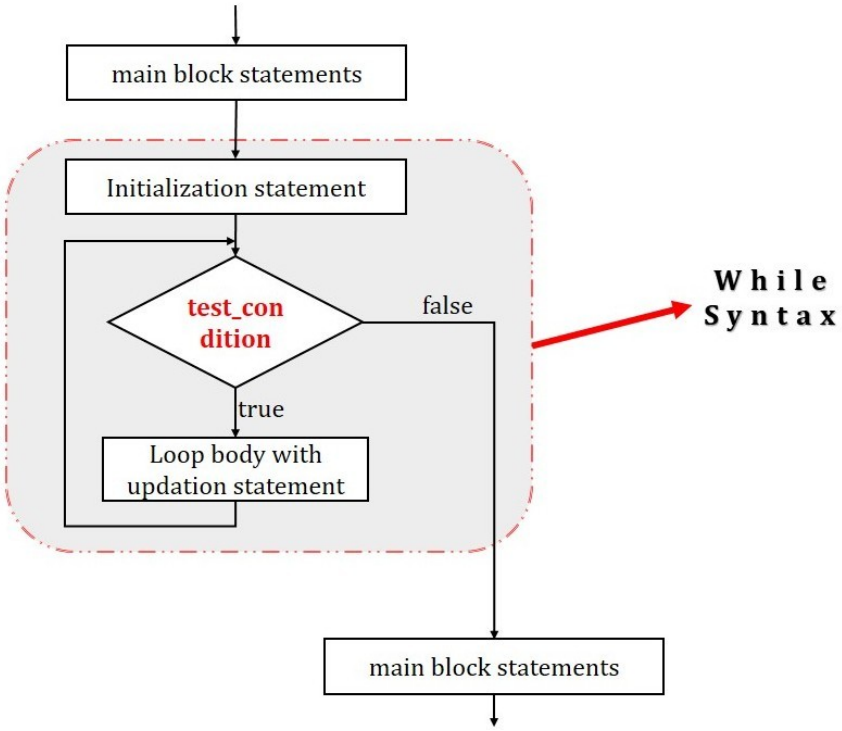
Component	Statement	Explanation
Initialization	i = 1	Assigns the starting value to the loop variable.
Test Condition	i <= 5	Checks whether the loop should continue.
Updation	i++	Increases the loop variable by 1 after each iteration.

i. while():

- ✓ The while loop in C executes a block of code repeatedly as long as the specified condition is true.
- ✓ ***Syntax:***

```
// optional main block statement;
initialization statement;
while(<test_condition>){
    // code to execute, if the test_condition is true;
    updation statement;
}
// optional main block statements;
```

- ✓ ***Flow Representation:***



- ✓ ***Example:***

```
// program to display N natural numbers
#include <stdio.h>
int main() {
   int i,n;
   printf("Enter an integer value for n:\n");
   scanf("%d",&n);
   i=1;                    /*initialization*/
   while (i <= n)          /*test_condition*/
{
      printf("Number: %d\n", i);
```

```
        i++;                /*updation*/
    }
    return 0;
}
```

Output:

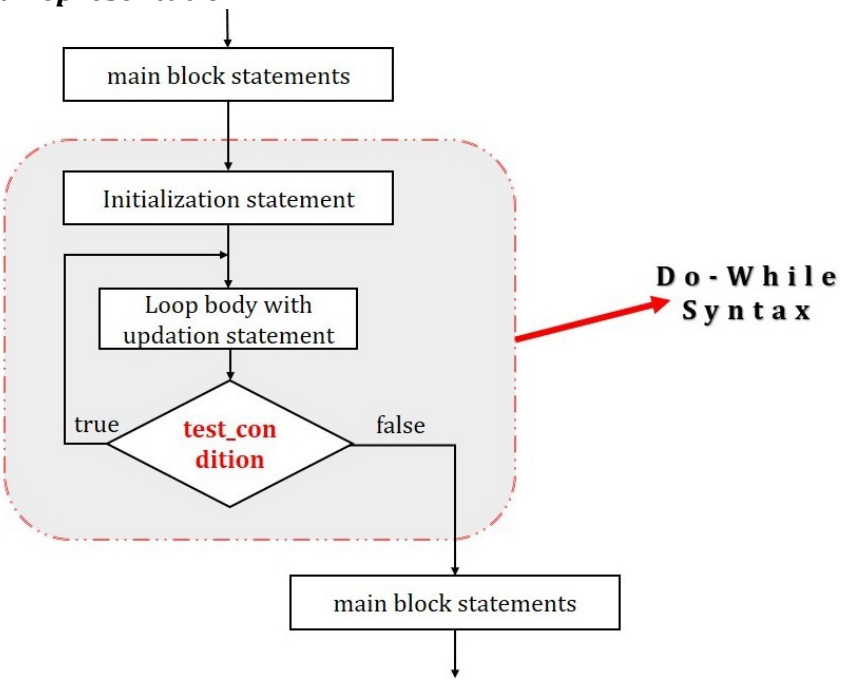
```
Enter an integer value for n: 5
Number: 1
Number: 2
Number: 3
Number: 4
Number: 5
```

ii. *do-while():*

- ✓ The do-while loop in C executes a block of code at least once and then continues to execute as long as the condition is true.
- ✓ Here, do have a block of statements and while should end with semicolon.
- ✓ ***Syntax:***

```
// optional main block statements;
initialization statement;
do{
    // code to execute;
    updation statement;
}while(<test_condition>);
// optional  main block statements;
```

- ✓ ***Flow Representation:***



- ✓ ***Example:***

```
// program to display N natural numbers
#include <stdio.h>
int main() {
   int i,n;
   printf("Enter an integer value for n:\n");
   scanf("%d",&n);
   i=1;                    /*initialization*/
   do {
      printf("Number: %d\n", i);
      i++;                 /*updation*/
   } while (i <= n);       /*test_condition*/
   return 0;
}
```

Output:

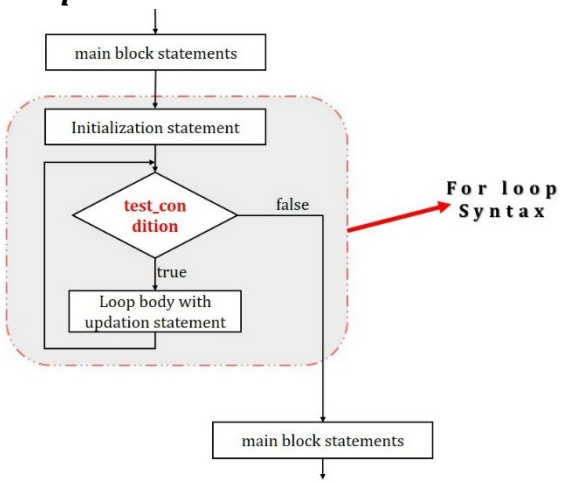
```
Enter an integer value for n: 5
Number: 1
Number: 2
Number: 3
Number: 4
Number: 5
```

iii. for():

- ✓ The for loop in C executes a block of code a specific number of times, controlled by an initialization, condition, and increment/decrement.
- ✓ ***Syntax:***

```
//optional main block statements
for (<initialization>; <test_condition>; <updation>) {
   // Code to execute in each iteration
}
//optional main block statements
```

- ✓ ***Flow Representation:***



✓ ***Example:***

```
// program to print N natural numbers
#include <stdio.h>
int main() {
   int i,n;
   printf("Enter an integer value for N:\n");
   scanf("%d",&n);
   for (i = 1; i <= n; i++) {
      printf("Number: %d\n", i);
   }
   return 0;
}
```

Output:

```
Enter an integer value for n: 5
Number: 1
Number: 2
Number: 3
Number: 4
Number: 5
```

➢ **Comparison Table:**

Feature	for Loop	while Loop	do-while Loop
Initialization	Before loop body, generally in loop header	Before loop	Before loop
Test Condition	Before loop body	Before loop body	After loop body
Updation	Generally in loop header	Generally inside loop body	Generally inside loop body
Minimum Executions	0	0	1
Loop Type	Entry-controlled	Entry-controlled	Exit-controlled

❒ **Important Points:**

- ✓ *Initialization:* Specifies where the loop starts.
- ✓ *Test Condition:* Specifies how long the loop continues.
- ✓ *Updation:* Specifies how the loop control variable changes during repetition.
- ✓ Initialization determines the starting point of a loop and this executes first time only.
- ✓ Test condition determines whether the loop should continue or terminate.
- ✓ In a for loop, updation is normally executed after each completed iteration.
- ✓ A while and for loop can execute zero or more times.
- ✓ A do-while loop executes at least once, even if its condition is initially false.

- ✓ A missing or incorrect updation may cause an infinite loop.
- ✓ In C, all three expressions in the for loop header are optional, but the two semicolons are mandatory.

iv. Jumping Control Statements

Jumping statements in C are used to transfer control of the program from one point to another. These statements enable conditional or unconditional jumps, allowing flexibility in program flow. There are four main jump control statements in C language:

i. break:

- ✓ The break statement in C terminates the nearest enclosing loop or switch statement and transfers control to the next statement after it.
- ✓ ***Syntax:***

```
break;
```

- ✓ ***Example:***

```
// program to demonstrate break statement
#include <stdio.h>
int main() {
        int i,n;
        printf("Enter an integer for n:");
        scanf("%d",&n);
          for (i = 1; i <= n; i++) {
            if (i == 3) {
              break;
            }
            printf("Number: %d\n", i);
          }
  return 0;
}
```

Output:

```
Enter an integer for n: 5
Number: 1
Number: 2
```

ii. continue:

- ✓ The continue statement in C skips the current iteration of the loop and moves to the next iteration.
- ✓ ***Syntax:***

```
continue;
```

- ✓ ***Example:***

```
// program to demonstrate continue statement
#include <stdio.h>
int main() {
        int i,n;
        printf(“Enter an integer for n:”);
        scanf(“%d”,&n);
          for (i = 1; i <= n; i++) {
            if (i == 3) {
              continue;
            }
            printf("Number: %d\n", i);
          }
  return 0;
}
```

Output:

```
Enter an integer for n: 5
Number: 1
Number: 2
Number: 4
Number: 5
```

iii. goto:

- ✓ The goto statement in C transfers control to a labeled statement within the program.
- ✓ ***Syntax:***

```
goto label;
...
label:
  // Code to execute
```

- ✓ ***Example:***

```
// program to demonstrate goto statement
#include <stdio.h>
int main() {
  int num;
  printf(“Enter an integer for num:”);
  scanf(“%d”,&num);
  if (num < 5) {
    goto small;
  }
  printf("Number is not small.\n");
```

```
    return 0;
    small:
    printf("Number is small.\n");
}
```

Output:

Enter an integer for num: 5
Number is small.

iv. return:

- ✓ The return statement in C exits the current function and optionally returns a value to the calling function.
- ✓ ***Syntax:***

```
return [<value>];
```

- ✓ ***Example:***

```
// program to demonstrate return statement
#include <stdio.h>
int add(int a, int b)            /* called function */
{
   return a + b;                 /* return statement to return sum of two values*/
}

int main() {
   int a,b,result;
   printf("Enter integer values for a,b:");
   scanf("%d%d",&a,&b);
   result = add(a,b);            /* calling function*/
   printf("Result: %d\n", result);
   return 0;
}
```

Output:

Enter integer values for a,b: 12

16

Result: 28

- **Comparison of Jumping Statements:**

Feature	break	continue	goto	return
Purpose	Terminates loop/switch	Skips current iteration	Jumps to a label	Terminates function
Used in loops	Yes	Yes	Yes	Yes
Used in switch	Yes	Only within an enclosing loop	Yes	Yes
Terminates loop	Yes	No	Depends on target	Yes, by exiting function
Terminates function	No	No	No	Yes
Requires label	No	No	Yes	No
Returns value	No	No	No	Optional
Jump type	Conditional or unconditional use	Conditional or unconditional use	Unconditional	Function exit